

DAT42-3

Machine Learning III — Unsupervised models

Clustering, dimensionality reduction, and evaluation without labels

Unsupervised learning is a business framing problem as much as a technical one.

Albert School — 22 hours

Contents

1	Unsupervised learning	2
2	K-means clustering	2
2.1	Implementation	2
2.2	Choosing k	3
2.3	Interpreting cluster profiles	3
2.4	Assumptions and limitations	3
3	Hierarchical clustering	4
3.1	The dendrogram	4
4	DBSCAN	4
5	PCA	5
5.1	Geometric interpretation	5
5.2	Explained variance ratio	5
6	t-SNE	5
7	Clustering evaluation	6
7.1	Silhouette score	6
7.2	Davies–Bouldin index	6
7.3	Limitations of internal metrics	6
8	Framing a business problem as unsupervised	6
9	Dimensionality reduction: visualization vs. downstream modelling	7

1 Unsupervised learning

In supervised learning (DAT42-1) every training example carries a label, and the model is trained to predict it. In **unsupervised learning** there are no labels. The algorithm is given only the feature vectors and must find structure in them: groups of similar points (clustering) or a lower-dimensional representation (dimensionality reduction).

Because there is no ground truth, there is no objective notion of a “correct” answer and no test-set accuracy to report. Whether an unsupervised result is good depends on whether it is useful for the business question being asked. Framing the problem and interpreting the output therefore matter as much as the algorithm.

This course assumes the supervised toolkit from DAT42-1 and the external mathematics (distance functions, matrix operations, variance).

2 K-means clustering

K-means partitions n points into k clusters, where k is chosen in advance. Each cluster is represented by its **centroid**, the mean of the points assigned to it. The algorithm minimises the total within-cluster squared distance to centroids (the **inertia**),

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2,$$

where μ_i is the centroid of cluster C_i .

2.1 Implementation

Lloyd’s algorithm alternates two steps until assignments stop changing:

1. **Assignment**: assign each point to the cluster whose centroid is nearest (Euclidean distance).
2. **Update**: recompute each centroid as the mean of the points assigned to it.

The result depends on the initial centroids; k-means++ initialisation spreads the initial centroids out and is the default. Run the algorithm several times (`n_init`) and keep the run with the lowest inertia.

```
from sklearn.cluster import KMeans

km = KMeans(n_clusters=4, n_init=10, random_state=0)
labels = km.fit_predict(X)  # cluster index for each point
centroids = km.cluster_centers_
```

K-means uses Euclidean distance, so features must be on comparable scales: standardise them first (`StandardScaler`, DAT42-1), or a single large-range feature will dominate the distance.

2.2 Choosing k

k is not given. Two standard methods:

- **Elbow method:** run k-means for a range of k , plot the inertia against k . Inertia always decreases as k grows, but the rate of decrease drops sharply at a point — the “elbow”. Choose k at the elbow.
- **Silhouette score:** compute the silhouette score (defined in Section 7) for a range of k and choose the k that maximises it.

The two need not agree, and the elbow is often ambiguous. The choice is a judgment, informed by these metrics and by whether the resulting clusters are interpretable.

2.3 Interpreting cluster profiles

A cluster index is meaningless on its own. To make the result actionable, describe each cluster by its **profile**: the mean (or typical) value of each feature within the cluster, compared against the overall average. Features on which a cluster departs strongly from the average characterise it.

```
import pandas as pd

df["cluster"] = labels
profiles = df.groupby("cluster").mean()  # feature means per cluster
```

Reading the profiles, give each cluster a business label (e.g. “high-spend, low-frequency customers”). This translation from centroid coordinates to a business description is the deliverable, not the labels themselves.

2.4 Assumptions and limitations

K-means assumes that clusters are:

- **roughly spherical** (isotropic) in feature space, because assignment uses plain Euclidean distance;
- **similar in size and density**;
- **separable by straight boundaries** — the boundary between two clusters is the perpendicular bisector of their centroids.

It also requires k to be fixed in advance and assigns every point to a cluster, so it has no notion of noise or outliers.

Consequently k-means breaks on datasets that violate these assumptions. Two concrete examples where it fails:

- **Concentric rings:** two clusters shaped as an inner disk and a surrounding ring. They are not linearly separable and not spherical, so k-means splits them across the diameter instead of recovering the rings.
- **Elongated or unequal clusters:** two long, parallel, stretched clusters, or one dense small cluster next to one sparse large cluster. K-means cuts them by distance to centroid and mixes points from different true groups.

Such shapes call for DBSCAN (Section 4) or hierarchical clustering (Section 3).

3 Hierarchical clustering

Hierarchical (agglomerative) clustering does not require k in advance. It builds a hierarchy of clusters:

1. Start with each point as its own cluster.
2. Repeatedly merge the two closest clusters.
3. Stop when a single cluster remains.

The distance between clusters is set by the **linkage**: **single** (nearest pair), **complete** (farthest pair), **average**, or **ward** (merge that least increases within-cluster variance). Ward with Euclidean distance is a common default.

3.1 The dendrogram

The sequence of merges is drawn as a **dendrogram**: a tree whose leaves are points and whose internal nodes are merges. The height of a merge is the distance between the two clusters joined. Points joined low in the tree are very similar; merges high up join already-dissimilar clusters.

To choose the number of clusters, cut the dendrogram with a horizontal line: the number of vertical lines it crosses is the number of clusters. Cutting across a large vertical gap (a tall edge with no merge over a wide height range) yields a stable, well-separated clustering.

```
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster

Z = linkage(X, method="ward")
dendrogram(Z)                                # inspect to choose the cut
labels = fcluster(Z, t=3, criterion="maxclust") # cut into 3 clusters
```

4 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups points by **density** rather than by distance to a centroid. It has two parameters: **eps** (a neighbourhood radius) and **min_samples** (the minimum number of points required within that radius).

A point is a **core point** if at least **min_samples** points lie within **eps** of it. Core points whose neighbourhoods reach each other belong to the same cluster, along with the non-core points on their borders. Points that are not reachable from any core point are labelled **noise** (cluster -1).

```
from sklearn.cluster import DBSCAN

db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X) # -1 marks noise
```

Differences from k-means:

- **Number of clusters:** discovered from the data, not fixed in advance.

- **Shape:** clusters are defined by connected dense regions, so DBSCAN recovers arbitrarily shaped clusters (e.g. the concentric rings of Section 2.4), which k-means cannot.
- **Noise:** points in low-density regions are left unassigned instead of being forced into a cluster.

DBSCAN is sensitive to `eps` and `min_samples`, and struggles when clusters have very different densities (a single `eps` cannot fit all of them). Features must be scaled, since `eps` is a distance.

5 PCA

Principal Component Analysis (PCA) reduces dimensionality by projecting the data onto a few directions that capture the most variance.

5.1 Geometric interpretation

Centre the data (subtract the mean of each feature). The **first principal component** is the direction in feature space along which the data varies most; the **second** is the direction of greatest remaining variance that is orthogonal to the first; and so on. The principal components are the eigenvectors of the data's covariance matrix, and each one's eigenvalue is the variance of the data along it.

Projecting the points onto the first few components gives a lower-dimensional representation that preserves as much variance as possible. Each component is a fixed linear combination of the original features; reading its coefficients (loadings) tells you which original features it represents.

5.2 Explained variance ratio

The **explained variance ratio** of a component is its eigenvalue divided by the sum of all eigenvalues — the fraction of total variance it captures. Plotting these ratios (a scree plot), or their cumulative sum, shows how many components are needed to retain a target fraction of the variance (e.g. 95%).

```
from sklearn.decomposition import PCA

pca = PCA()
scores = pca.fit_transform(X_scaled)          # coordinates in PC space
print(pca.explained_variance_ratio_)          # fraction of variance per component
print(pca.explained_variance_ratio_.cumsum())
```

Standardise features before PCA: because PCA maximises variance, an unscaled high-variance feature would dominate the components for reasons of units alone.

6 t-SNE

t-SNE (t-distributed Stochastic Neighbour Embedding) is a **visualization** method. It maps high-dimensional points to two or three dimensions so that points which are close in the original space tend to stay close in the map, producing a plot in which clusters are visually apparent.

```
from sklearn.manifold import TSNE

emb = TSNE(n_components=2, perplexity=30, random_state=0).fit_transform(X)
```

t-SNE preserves **local** neighbourhoods but not global geometry. Consequently its distances are not directly interpretable:

- the distance between two clusters on the map does not reflect their true separation;
- the size and density of a blob on the map do not reflect the true size or spread of that cluster;
- the result depends on the `perplexity` parameter and on the random seed.

Use a t-SNE plot to see whether structure exists, not to measure it. It is a visualization tool, not a basis for downstream modelling (Section 9).

7 Clustering evaluation

With no labels, clustering is judged by **internal metrics** — measures computed from the data and the cluster assignments alone.

7.1 Silhouette score

For a point i , let $a(i)$ be its mean distance to the other points in its own cluster, and $b(i)$ the mean distance to the points of the nearest other cluster. Its silhouette is

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))},$$

which ranges from -1 to 1 . A value near 1 means the point sits well inside its cluster and far from others; near 0 means it lies on a boundary; negative means it is probably mis-assigned. The **silhouette score** is the mean of $s(i)$ over all points; higher is better.

7.2 Davies–Bouldin index

The **Davies–Bouldin index** averages, over all clusters, the worst-case ratio of within-cluster spread to between-cluster separation for each cluster against the others. Lower is better, with 0 the ideal. Unlike the silhouette, smaller values indicate better-defined clusters.

```
from sklearn.metrics import silhouette_score, davies_bouldin_score
```

```
silhouette_score(X, labels)    # higher is better
davies_bouldin_score(X, labels) # lower is better
```

7.3 Limitations of internal metrics

Internal metrics measure geometric properties — compactness and separation — not correctness, because there is no ground truth to compare against. They carry built-in assumptions: silhouette and Davies–Bouldin both reward compact, well-separated, roughly convex clusters, so they favour k-means-style solutions and can score a correct DBSCAN clustering of non-convex shapes poorly. A high score does not mean the clustering is **useful**; only that it is geometrically clean. The final test of a clustering is whether its clusters are interpretable and actionable for the business problem, not its internal-metric value.

8 Framing a business problem as unsupervised

Unsupervised learning is a framing problem as much as a technical one. A workable project runs:

1. **Frame:** state the business question and decide it is genuinely unsupervised — there is no label to predict, the goal is to discover groups or structure (e.g. “what natural segments exist among our customers?”).
2. **Prepare:** select and scale the features that are relevant to that question. Feature choice determines what the clusters will mean.
3. **Apply:** choose an algorithm suited to the expected structure (k-means for compact segments, DBSCAN for arbitrary shapes and noise, hierarchical when a hierarchy or no fixed k is wanted), and select its parameters using the methods above.
4. **Evaluate:** check the result with internal metrics and, above all, by inspecting cluster profiles for interpretability.

5. **Report:** deliver **actionable cluster descriptions** — for each cluster, a business-meaningful profile and what should be done with it.

The deliverable for this course is such a written report: a business problem framed as an unsupervised task, an appropriate algorithm applied, and clusters described in terms a decision-maker can act on.

9 Dimensionality reduction: visualization vs. downstream modelling

Dimensionality reduction serves two distinct purposes, and the two must not be confused.

- **For visualization:** reduce to two or three dimensions to plot the data and inspect structure. t-SNE is suitable here, but its output is only for the eye — its coordinates are not meaningful features (Section 7) and must not be fed into a model.
- **For downstream modelling:** reduce dimensionality to produce features for a later model — to remove redundancy, reduce noise, or speed up training. This needs a method whose output is a stable, meaningful transformation that can be applied to new data. PCA fits this use: it defines a fixed linear projection that can be applied unchanged to future points, and its components retain a defined fraction of the variance.

In short: use t-SNE to **see**, use PCA to **model**. Using t-SNE coordinates as model inputs, or expecting PCA to give the cleanest possible 2-D picture, mistakes one use for the other.